SWEN20003 OPEN-BOOK EXAM CHEAT SHEET & REFERENCE MANUAL

Part I: Visual Topic Tree Index | Part II: OOP Lookup Tables, UML Rules, Design Patterns & Java Templates

IN-SCOPE (Core Exam Syllabus)	OUT-OF-SCOPE (Never Tested)
• Java Memory, String Pool, Wrappers, Immutability & Defensive Copy • Inheritance, Polymorphism, Overriding vs Overloading Dispatch • Abstract Classes vs Interfaces, Upcasting/Downcasting • Reading & Drawing UML Class Diagrams (Multiplicity, Diamonds) • SOLID Principles (SRP, OCP, LSP, ISP, DIP) & Code Critique • GoF Patterns: Strategy, Singleton, Observer, Template Method, State • Generics (<T extends X>), ArrayList, HashMap, Comparable/Comparator • Checked vs Unchecked Exceptions, Streams, Automated Testing	• Bagel / JavaFX Game Engine API Method Signatures • Gradle Build Scripts (build.gradle.kts) & Module Info Syntax • Complex Concurrency / Thread Locks (beyond Singleton synchronized) • Bytecode / JVM Garbage Collector Internal Algorithms • Network Socket Programming or SQL Database JDBC Drivers • Memorizing Java Standard Library Appendix Docs (provided in exam)

PART I · VISUAL TOPIC TREE INDEX

```
[!!!] = Core / Must-know    [!! ] = Important / Frequent    [!  ] = Basic / Edge case

[0.0] SWEN20003 OBJECT-ORIENTED SOFTWARE DEVELOPMENT (JAVA)
│
├── [1.0] Java Memory Model, Strings, Arrays & Immutability
│   ├── [1.1] Primitives, References & String Pool
│   │   ├── [1.1.1] Stack primitives vs Heap reference variables         [!! ]
│   │   ├── [1.1.2] Reference equality (==) vs value equality (.equals)  [!!!]
│   │   ├── [1.1.3] String literal pool vs new String() & StringBuilder  [!! ]
│   │   └── [1.1.4] Wrapper classes, Autoboxing/Unboxing & Integer cache [!! ]
│   │
│   ├── [1.2] Encapsulation, Privacy Leaks & Immutability
│   │   ├── [1.2.1] Access modifiers (private, package, protected, pub)  [!!!]
│   │   ├── [1.2.2] Information hiding & constructor/getter privacy leak [!!!]
│   │   ├── [1.2.3] Defensive copying (deep copy in constructor/getter)  [!!!]
│   │   └── [1.2.4] Complete Immutable Class 5-step recipe               [!!!]
│   │
│   └── [1.3] Static vs Instance & Parameter Passing
│       ├── [1.3.1] Pass-by-value semantics for primitives & references  [!!!]
│       └── [1.3.2] Static fields/methods, final constants & utility cls [!! ]
│
├── [2.0] Inheritance, Polymorphism, Binding & Abstract Types
│   ├── [2.1] Inheritance Mechanics & Constructor Chaining
│   │   ├── [2.1.1] extends keyword, single inheritance & super() rules  [!!!]
│   │   ├── [2.1.2] Upcasting (implicit safe) vs Downcasting + instanceof[!!!]
│   │   └── [2.1.3] Object cosmic superclass (toString, equals, hashCode)[!!!]
│   │
│   ├── [2.2] Polymorphism, Overriding vs Overloading & Dispatch
│   │   ├── [2.2.1] Static (declared) type vs Dynamic (actual) type      [!!!]
│   │   ├── [2.2.2] Method Overriding (@Override) & dynamic binding      [!!!]
│   │   ├── [2.2.3] Method Overloading & compile-time static binding     [!!!]
│   │   └── [2.2.4] Field hiding & static method hiding (compile-time)   [!! ]
│   │
│   └── [2.3] Abstract Classes vs Interfaces
│       ├── [2.3.1] Abstract classes & abstract method contracts         [!!!]
│       ├── [2.3.2] Interfaces (implements, multiple inheritance of type)[!!!]
│       └── [2.3.3] Abstract Class ("IS-A") vs Interface ("CAN-DO")      [!!!]
│
├── [3.0] UML Class Diagrams & OO Design Principles
│   ├── [3.1] UML Class Box & Visibility Notation
│   │   ├── [3.1.1] 3-compartment box: Name, Attributes, Operations      [!!!]
│   │   └── [3.1.2] Visibility symbols (-, +, #, ~) & static underline   [!!!]
│   │
│   ├── [3.2] Six Core UML Class Relationships & Multiplicities
│   │   ├── [3.2.1] Association & Navigability arrow + Multiplicity      [!!!]
│   │   ├── [3.2.2] Aggregation (Hollow Diamond) weak whole-part         [!!!]
│   │   ├── [3.2.3] Composition (Filled Solid Diamond) strong lifecycle  [!!!]
│   │   ├── [3.2.4] Inheritance / Generalization (Solid line, hollow tri)[!!!]
│   │   ├── [3.2.5] Realization / Interface (Dashed line, hollow tri)    [!!!]
│   │   └── [3.2.6] Dependency (Dashed line, open arrow)                 [!! ]
│   │
│   └── [3.3] Coupling, Cohesion & Delegation
│       ├── [3.3.1] Low Coupling & High Cohesion design evaluation       [!!!]
│       └── [3.3.2] Favor Composition/Delegation over Inheritance        [!!!]
│
├── [4.0] SOLID Principles & Gang of Four (GoF) Design Patterns
│   ├── [4.1] SOLID Design Principles
```

```
│   │   ├── [4.1.1] Single Responsibility Principle (SRP)               [!!!]
│   │   ├── [4.1.2] Open/Closed Principle (OCP)                         [!!!]
│   │   ├── [4.1.3] Liskov Substitution Principle (LSP)                 [!! ]
│   │   ├── [4.1.4] Interface Segregation Principle (ISP)               [!! ]
│   │   └── [4.1.5] Dependency Inversion Principle (DIP)                [!! ]
│   │
│   └── [4.2] Gang of Four (GoF) Design Patterns
│       ├── [4.2.1] Singleton Pattern (Creational: 1 lazy instance)     [!!!]
│       ├── [4.2.2] Strategy Pattern (Behavioural: swappable algorithms) [!!!]
│       ├── [4.2.3] State Pattern (Behavioural: state-driven transitions)[!!!]
│       ├── [4.2.4] Observer Pattern (Behavioural: 1-to-many pub/sub)   [!!!]
│       ├── [4.2.5] Template Method Pattern (Behavioural: skeleton algo) [!!!]
│       └── [4.2.6] Factory / Adapter / Decorator pattern recognition   [!  ]
│
├── [5.0] Generics, Collections, Sorting & Streams
│   ├── [5.1] Parametric Polymorphism (Generics)
│   │   ├── [5.1.1] Generic classes/methods <T, K, V> & type safety     [!!!]
│   │   ├── [5.1.2] Bounded type parameters <T extends UpperBound>      [!!!]
│   │   └── [5.1.3] Wildcards (<?>, <? extends T>, <? super T>) & erasure[!! ]
│   │
│   ├── [5.2] Java Collections Framework (List, Set, Map, Queue)
│   │   ├── [5.2.1] ArrayList vs LinkedList & Iterator safe removal     [!!!]
│   │   ├── [5.2.2] HashMap<K,V> & nested Map<K, ArrayList<V>> patterns  [!!!]
│   │   └── [5.2.3] HashSet/TreeSet & equals()/hashCode() contract      [!! ]
│   │
│   └── [5.3] Object Comparison & Functional Streams
│       ├── [5.3.1] Comparable<T> (compareTo) vs Comparator<T> (compare) [!!!]
│       └── [5.3.2] Lambda expressions & Stream pipeline (filter/map/sort)[!! ]
│
└── [6.0] Exceptions, File I/O, Event-Driven GUI & Testing
    ├── [6.1] Exception Handling Architecture
    │   ├── [6.1.1] Throwable hierarchy: Checked vs Unchecked (Runtime)  [!!!]
    │   ├── [6.1.2] try-catch-finally execution order & multi-catch     [!!!]
    │   └── [6.1.3] try-with-resources (AutoCloseable) & custom Exception[!!!]
    │
    ├── [6.2] File I/O & Text Processing
    │   ├── [6.2.1] BufferedReader / FileReader / Scanner / PrintWriter  [!! ]
    │   └── [6.2.2] CSV parsing (.split(",")) & String formatting       [!! ]
    │
    └── [6.3] Event-Driven Programming & Automated Testing
        ├── [6.3.1] Event-driven loop, Listeners/Handlers & MVC decouple [!! ]
        └── [6.3.2] Automated Unit Testing (JUnit @Test, assertions)    [!!!]
```

PART II · OPEN-BOOK REFERENCE TABLES, UML RULES & JAVA TEMPLATES

[1.0] Java Memory Model, Encapsulation, Privacy Leaks & Immutability

Index & Topic	Java Rule / Mechanism	Critical Exam Trap / Fix
[1.1.2] == vs .equals()	• == compares primitive values OR memory addresses (references) • .equals() compares logical object content (must override Object.equals)	NEVER use == to compare Strings or Wrapper objects! Always use s1.equals(s2)
[1.1.3] String Pool	• String s1 = "hi"; uses shared String Constant Pool on heap • String s2 = new String("hi"); forces a new distinct heap object	s1 == s2 is FALSE, but s1.equals(s2) is TRUE. Use StringBuilder in loops to avoid O(n^2) String allocations
[1.1.4] Wrapper Cache	• Autoboxing: Integer a = 100; (calls Integer.valueOf(100)) • Java caches Integer instances in range [-128, 127]	Integer a=100, b=100 -> a==b is TRUE (cached). Integer x=200, y=200 -> x==y is FALSE (new heap objects)!
[1.2.1] Access Modifiers	• private: same class only • (default/package): same package • protected: same package + subclasses anywhere • public: everywhere	Encapsulation rule: ALL instance fields must be private (or private final)
[1.2.2] Privacy Leak & Defensive Copy	• Leak 1 (Constructor): this.list = list; (caller keeps reference to mutable list) • Leak 2 (Getter): return this.list; (caller mutates internal list via getter)	• Fix 1 (Constructor): this.list = new ArrayList<>(list); • Fix 2 (Getter): return new ArrayList<>(this.list); (or Collections.unmodifiableList)
[1.2.4] Immutable Class 5-Step Checklist	1. Declare class as public final class ... (prevents subclassing) 2. Declare all fields as private final 3. Do NOT provide any setter (mutator) methods	4. In Constructor: perform deep defensive copy of all mutable input parameters 5. In Getters: return deep defensive copies of all mutable fields (String/primitives are already immutable, so direct return is safe)

```
/* [1.2.4] Complete Exam-Ready Immutable Class Template with Defensive Copying */
public final class ImmutablePerson {
    private final String name;                 // String is already immutable -> safe
    private final List<String> hobbies;        // List is mutable -> MUST defensive copy!

    public ImmutablePerson(String name, List<String> hobbies) {
        this.name = name;
        this.hobbies = new ArrayList<>(hobbies);   // Line A Fix: Defensive copy on input!
    }
    public String getName() { return this.name; }
    public List<String> getHobbies() {
        return new ArrayList<>(this.hobbies);      // Line B Fix: Defensive copy on output!
    }
}
```

[2.0] Inheritance, Polymorphism, Overriding vs Overloading & Abstract Types

Index & Feature	Compile-Time (Static) Rule	Run-Time (Dynamic) Execution Rule
[2.2.1] Static vs Dynamic Type: Base b = new Sub();	• Static (Declared) Type = Base • Compiler checks ONLY Base class to verify if method/field exists	• Dynamic (Actual Heap) Type = Sub • JVM dispatches overridden instance methods based on Sub at runtime
[2.2.2] Method Overriding (@Override)	• Exact same method name AND exact same parameter types in subclass • Return type must be same (or covariant subtype); visibility cannot be narrower	• DYNAMIC BINDING (Late Binding): JVM runs the Subclass implementation based on actual heap object type
[2.2.3] Method Overloading (Classic Exam Trap!)	• Same method name, DIFFERENT parameter list (type, count, or order) • Can occur in same class OR between superclass and subclass	• STATIC BINDING (Early Binding): Compiler locks the exact method signature at compile time using ONLY the Static Declared Type of the caller variable and argument static types!
[2.2.4] Static Methods & Instance Fields	• Cannot override static methods or instance fields (only hides them)	• STATIC BINDING: Resolved strictly at compile time using Static Declared Type
[2.1.1] Constructor Chain	• First line of subclass constructor MUST be super(...) or this(...) • If omitted, compiler automatically inserts implicit super() (no-arg)	• If superclass has NO no-arg constructor, subclass MUST explicitly call super(args) on Line 1 or compilation fails!

Index & Feature	Compile-Time (Static) Rule	Run-Time (Dynamic) Execution Rule
[2.1.2] Casting & instanceof	• Upcasting: Base b = new Sub(); (Always safe, implicit) • Downcasting: Sub s = (Sub) b; (Requires explicit cast)	• If actual object is NOT a Sub, (Sub) b throws ClassCastException at runtime! • Always guard with: if (b instanceof Sub) { Sub s = (Sub) b; }

[2.3.3] Dimension	Abstract Class (abstract class A)	Interface (interface I)
Core Relationship	"IS-A" identity & shared code hierarchy (e.g., Staff -> Doctor, Receptionist)	"CAN-DO" capability contract across unrelated classes (e.g., Comparable, Attackable)
Multiple Inheritance	Single inheritance ONLY: a class can extend at most ONE class	Multiple implementation: a class can implement MANY interfaces (implements A, B, C)
State & Fields	Can hold non-final, private/protected instance variables (mutable state)	NO instance fields! All fields are implicitly public static final constants
Constructors & Methods	Has constructors (called via super()); mixes concrete + abstract methods	NO constructors; abstract methods (plus Java 8+ default/static utility methods)

[3.0] UML Class Diagrams, Relationships & OO Design Quality

[3.2] UML Relationship	Visual Arrow / Symbol Style	Meaning, Lifecycle & Java Code Mapping
[3.2.1] Association	Solid line with open arrow (A □□□□□□> B)	• "HAS-A" / knows about: Class A holds a reference field to Class B (private B b;) • Multiplicity placed at target end: 1 (exact), 0..1 (optional), * or 0..* (many), 1..* (at least 1)
[3.2.2] Aggregation (Weak Whole-Part)	Solid line with HOLLOW WHITE diamond at Whole (Whole □□□□□□□ Part)	• "HAS-A" collection where Part CAN exist independently if Whole is destroyed • Exam Example: Department □□□□□□□ Professor (if Department closes, Professor still lives) • Java Code: Whole receives Part via constructor/setter; holds List<Part>
[3.2.3] Composition (Strong Whole-Part)	Solid line with FILLED BLACK diamond at Whole (Whole □□□□□□□ Part)	• Exclusive ownership & LIFECYCLE DEPENDENCY: if Whole is destroyed, Parts cease to exist! • Exam Example: University □□□□□□□ Department (or Car □□□□□□□ Engine) • Java Code: Whole instantiates Part internally (this.part = new Part()) and never leaks it
[3.2.4] Inheritance (Generalization)	Solid line with HOLLOW TRIANGLE pointing to Super (Sub □□□□□□□ Super)	• "IS-A" relationship between classes (class Sub extends Super) • Arrow ALWAYS points UP to the parent/superclass!
[3.2.5] Realization (Implementation)	DASHED line with HOLLOW TRIANGLE pointing to Interface (Class - - - -□ <<interface>>)	• "CAN-DO" implementation (class ConcreteStrategy implements Strategy) • Interface box is marked with guillemets <<interface>> above the name
[3.2.6] Dependency	DASHED line with open arrow (A - - - -> B)	• "USES-A" temporary relationship: A takes B as a method parameter, local var, or static call

[3.1 / 3.3] Concept	Notation / Definition	Exam Justification & Application
[3.1.2] UML Visibility	• - private • + public • # protected • ~ package • Underlined = static • Italics / {abstract} = abstract	Attribute format: - name : Type = defaultValue Method format: + methodName(param : Type) : ReturnType
[3.3.1] Coupling vs Cohesion (2017 Q1d)	• Coupling: Degree of interdependence between different classes • Cohesion: Degree to which elements inside ONE class belong together	• Goal: LOW Coupling + HIGH Cohesion • Low coupling makes classes easy to modify/reuse independently; high cohesion obeys SRP
[3.3.2] Composition / Delegation > Inheritance	• Delegation: Object hands over a task to a helper/strategy object field • Inheritance is fixed at compile time; Composition swaps behaviour at runtime	• Avoids class explosion (e.g., MagicFlyingHero, MeleeFlyingHero) • Changes to superclass implementation do not break subclasses (fragile base class)

[4.0] SOLID Principles & Gang of Four (GoF) Design Patterns

[4.1] SOLID Principle	Core Definition	Code Smell (Violation) -> Refactoring Solution
[4.1.1] S — Single Responsibility (SRP)	A class should have one, and only one, reason to change (one cohesive responsibility).	• Smell: OrderProcessor calculates discounts, executes SQL, AND sends emails. • Fix: Split into DiscountCalculator, OrderRepository, and EmailService.
[4.1.2] O — Open/Closed Principle (OCP)	Software entities should be OPEN for extension, but CLOSED for modification.	• Smell: if (type.equals("VIP")) ... else if (type.equals("STUDENT")) inside processor. • Fix: Introduce DiscountStrategy interface; add new classes without editing OrderProcessor.
[4.1.3] L — Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without altering correctness.	• Smell: Square extends Rectangle (setting width mutates height, breaking area tests) or subclass throws UnsupportedOperationException. • Fix: Extract common Shape interface instead of forcing invalid inheritance.
[4.1.4] I — Interface Segregation (ISP)	Clients should not be forced to depend on fat interfaces with methods they do not use.	• Smell: Fat Worker interface with work() and eat() forces Robot to implement dummy eat(). • Fix: Split into smaller role interfaces Workable and Feedable.
[4.1.5] D — Dependency Inversion (DIP)	High-level modules should not depend on low-level concrete classes; both should depend on abstractions.	• Smell: OrderProcessor directly calls new MySQLDatabase() or static EmailSender. • Fix: Depend on DatabaseInterface / NotificationService injected via constructor.

[4.2] GoF Pattern	Family & Intent	Participants & Structure	When to Select in Exam / Key Benefit
[4.2.1] Singleton	Creational: Ensure exactly ONE instance + global access point	• Singleton class with: 1. private static Singleton instance; 2. private Singleton() {} 3. public static getInstance()	• Logger, Database Connection Pool, Config Manager • Better than public static field: lazy initialization, prevents overwriting (instance = null), encapsulates creation logic
[4.2.2] Strategy	Behavioural: Encapsulate family of interchangeable algorithms (1-to-1)	• Context (e.g., Course, Character) • Strategy Interface (e.g., DisplayStrategy) • ConcreteStrategies (SortByName, SortByGPA)	• Switching attack/sorting/payment algorithms dynamically at runtime via setStrategy() • Satisfies OCP & replaces conditional if/else blocks
[4.2.3] State	Behavioural: Object alters behaviour when internal state changes	• Context (e.g., Mokepon) • State Interface (ConditionState) • ConcreteStates (Asleep, Poisoned, Normal)	• Entity has distinct states (Asleep/Poisoned/Confused) that change turn behaviour AND transition dynamically during battle
[4.2.4] Observer	Behavioural: 1-to-Many publish-subscribe automatic notification	• Subject (holds List<Observer>, add/remove/notify) • Observer Interface (update / onChanged) • ConcreteObservers (Timetable, Billing, LMS)	• WeatherStation / EnrolmentSystem updates multiple displays/services without knowing their concrete classes (loose coupling, OCP)
[4.2.5] Template Method	Behavioural: Define algorithm skeleton in base class; defer steps to subclasses	• AbstractClass (has final templateMethod() + abstract step methods) • ConcreteSubclasses (ExamReport, RPGGame)	• Two classes share identical step sequence (open -> header -> body -> close) or game loop (input -> update -> render), differing only in specific steps

```
/* [4.2.1] Singleton Template */
public class Logger {
    private static Logger instance = null;
    private Logger() {} // Private!
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
}
```

```
/* [4.2.2] Strategy Pattern Template */
public interface AttackStrategy {
    int attack();
}
public class Hero {
    private AttackStrategy strategy;
    public void setStrategy(AttackStrategy s) {
        this.strategy = s;
    }
    public int performAttack() {
        return strategy.attack(); // Delegate!
    }
}
```

```
/* [4.2.4] Observer Subject Templa
public class Subject {
    private List<Observer> obs = n
    public void addObserver(Observ
    public void notifyObservers(St
        for (Observer o : obs) {
            o.update(data);
        }
    }
}
public interface Observer { void u
```

[5.0] Generics, Collections, Comparable vs Comparator & Streams

Index & Topic	Syntax / Contract	Exam Coding Pattern & Pitfalls

[5.1.1] Generic Class	public class Pair<T, U> {
	private T first; private U second;
	}
		Instantiate with diamond operator: Pair<String, Integer> p = new Pair<>("A", 1);
		Type arguments MUST be Reference types (use Integer/Double, NEVER int/double)

[5.1.2] Bounded Generics (2017 Exam Q6)	public class CategorisedMap<K, V, C extends Categoriser<V, K>>	Always use keyword 'extends' for upper bounds in generic headers — even when Categoriser is an interface! Inside class, can safely call categoriser.categorise(val)

[5.2.1] Iterator Removal	Iterator<Student> it = list.iterator();
	while (it.hasNext()) {
	if (it.next().getId().equals(id)) it.remove();
	}
		NEVER call list.remove(x) inside a for-each loop (for (Student s : list)) -> throws ConcurrentModificationException! Use Iterator.remove() or index loop

[5.2.2] Nested HashMap (2016 Q3 / 2017 Q6)	HashMap<K, ArrayList<V>> map = new HashMap<>();
	if (!map.containsKey(k)) map.put(k, new ArrayList<>());
	map.get(k).add(val);
		When mapping one key to multiple values, check !map.containsKey(k) first and initialize the inner ArrayList<V> before calling .get(k).add(val) to avoid NullPointerException!

[5.3.1] Comparable<T> vs Comparator<T>	• Comparable<T> (Natural Order inside class): public int compareTo(Card other) • Comparator<T> (External Strategy class/lambda): public int compare(Card c1, Card c2)	• Return sign rule: • Ascending order: return this.val - other.val; (or Double.compare(a, b)) • Descending order (2017 Q4a): return other.getPoints() - this.getPoints(); • Sort call: Collections.sort(list); OR list.sort(comparator);
[5.3.2] Java Streams (2017 Exam Q1f)	list.stream() .filter(m -> m.isActive()) .sorted((a, b) -> a.getScore() - b.getScore()) .collect(Collectors.toList());	• .stream(): creates stream from collection without mutating original list • .filter(Predicate): keeps only elements where lambda returns true • .map(Function): transforms elements • .sorted(Comparator): orders stream • .collect(Collectors.toList()): terminal op packaging results into a new List

[6.0] Exceptions, File I/O & Automated Software Testing

Index & Topic	Rules & Mechanics	Exam Answers & Best Practices
[6.1.1] Checked vs Unchecked Exceptions	• Checked (extends Exception, not RuntimeException): IOException, FileNotFoundException, SQLException. Compiler forces try-catch or 'throws' declaration. • Unchecked (extends RuntimeException): NullPointerException, IndexOutOfBoundsException, IllegalArgumentException, ClassCastException.	Use Checked exceptions for recoverable external environment failures (missing file); use Unchecked exceptions for programming/logic bugs (null args, bad index).
[6.1.2] Catch Ordering & Finally Block	• Catch blocks MUST be ordered from MOST SPECIFIC (subclass) to MOST GENERAL (superclass), e.g., catch (FileNotFoundException) BEFORE catch (IOException). • finally block ALWAYS executes whether exception is thrown or not.	Putting catch (Exception e) first causes a compile-time 'Unreachable catch block' error! Use try (BufferedReader br = new BufferedReader(...)) {} for automatic resource closing.
[6.3.2] Automated Testing (2017 Q1e & Q3b)	• Automated Testing: Writing code/scripts (e.g. JUnit @Test) that automatically execute target methods with test inputs and assert expected outputs.	• Two reasons preferred over manual testing: 1. Fast, repeatable regression testing on every code change without human effort. 2. Eliminates human error/fatigue and checks edge/boundary cases systematically. • Exam Test Case Format: (1) Component/Behaviour tested, (2) Concrete Input, (3) Expected Output/State.